

Fraction of Time MPI-SPPY spends in Python

David L Woodruff
Graduate School of Management

University of California Davis
Davis CA 95616 USA

July 29, 2026

When considering applications in practice, or when comparing to other packages, a question arises concerning the fraction of time that `mpi-sppy` spends “in Python” as opposed to compiled code written in other languages such as C and Fortran. Since solvers, `numpy`, and MPI are all in the latter category, *a priori* one expects that the fraction spent in Python will be small.

We have used the tool called *scalene*, which is designed to attribute time to Python vs native (it can estimate time spent in compiled code called from Python, which it calls “native”). It works by sampling.

The short answer is that there is no single number: the fraction depends almost entirely on how much work the solver does per subproblem. When the subproblems are substantial, as in the `sslp_15_45` instances, about 9% of the time is spent in Python, which matches the *a priori* expectation. When the subproblems are small enough that the solver returns almost immediately, as in `farmer`, the majority of the time is spent in Python — 76% in the most extreme case measured here. The crossover is not subtle and it is not a sampling artifact.

The reason is that each subproblem solve costs a roughly fixed amount of Python work regardless of how hard the subproblem is: the proximal objective has to be rebuilt and handed to the solver, and the solution has to be read back. That cost is amortized over the solver’s work, so it is invisible when the solver is busy for a while and dominant when the solver is not.

The practical consequences are:

- If your subproblems keep the solver busy, you cannot get much speed-up just by using another language. Maybe another language will make it easier for you to do something sophisticated and algorithmic.
- If your subproblems are small, the per-solve Python cost is what you are paying for, and the fix is to give the solver more work per call rather than to rewrite anything. Bundling is the obvious lever, and it is examined below.

1 Method

Every case runs the same three cylinders — a PH hub, a lagrangian spoke and an `xhatshuffle` spoke, one MPI rank each — so that only the model and the instance change between cases. The convergence tolerance is set to zero so that runs end on the iteration limit rather than on a gap, which keeps the amount of work per case predictable.

Because *scalene* samples, every case is run three times and the tables report the mean over repetitions with the observed min–max range. The spread turns out to be small for runs of a minute or so (typically a few tenths of a percentage point) and much larger for the deliberately short `farmer3` case, which is why that case is kept: it shows what the numbers look like when there is not enough run time to sample properly.

The numbers are read out of *scalene*’s JSON profile rather than scraped from the output of `scalene view --cli`. Summing the per-line percentages in the JSON gives the same quantities at full precision, and avoids three problems with scraping the terminal output: it rounds each line to a whole percent,

its `--reduced` form omits low-usage lines so the sums undercount, and it is colourized, which silently broke the row-matching in the earlier version of this code. Scalene attributes 91–99% of wall time to some source line; the percentages below are shares of that attributed time, so the unattributed remainder is divided out rather than being silently credited to one of the buckets.

All runs are on a single machine with homogeneous cores, which avoids a difficulty that affected an earlier version of these experiments: on a machine with a mixture of fast and slow cores, ranks landing on slow cores showed inflated Python fractions.

System and software:

- `os=Linux 7.0.0-28-generic (x86_64)`
- `cpu_model=AMD Ryzen 7 7840HS`
- `cpu_physical_cores=8, cpu_logical=16, threads_per_core=2`
- `cpu_max_mhz=5137.9`
- `mem_total=14.31 GiB`
- `python=3.11.13, scalene=2.0.1, pyomo=6.9.5.dev0, mpi4py=4.1.0, gurobipy=13.0.2`
- solver interface: `gurobi_persistent, --max-solver-threads 2`

2 Results

Table 1 gives the split by case and Table 2 breaks the Python percentage out by cylinder. The cases are:

- **farmer3** — 3 scenarios, 100 iterations. Deliberately short: about two seconds unprofiled, five under scalene.
- **farmer60**, **farmer240** — 60 and 240 scenarios, 400 and 200 iterations. Many trivially small LP subproblems.
- **farmer240_bun10** — **farmer240** with ten scenarios gathered into each proper bundle, so 24 subproblems instead of 240.
- **sslp_15_45_10**, **sslp_15_45_15** — 10 and 15 scenarios, 50 iterations. MIP subproblems that keep the solver busy.
- **sslp_15_45_15_bun3** — **sslp_15_45_15** with three scenarios per bundle, so 5 subproblems instead of 15.
- **sslp_5_25_50** — 50 scenarios, 150 iterations. Many small MIP subproblems.

The ordering is by subproblem difficulty, not by instance size. The two **sslp_15_45** cases sit at 8.7% and 9.6% Python; every case whose subproblems are small sits far above that, from 58% for **sslp_5_25_50** up to 76% for **farmer240**. Note that **sslp_5_25_50** has more scenarios than **sslp_15_45_15** and yet spends six times the fraction of its time in Python: what matters is the size of each subproblem, not how many there are.

Looking at where the samples land makes the mechanism explicit. In both regimes essentially all of the time is attributed to one line, `spopt.py:337`, which is the Pyomo `solve` call. For **sslp_15_45_15** that line is 6.6% Python and 88.2% native — the solver is doing the work. For **farmer60** the same line is 48.2% Python and 5.8% native, and a further 9.4% of the run is Python time in `spopt.py:267`, where the proximal objective is handed to the solver on each iteration. That objective work scales with the size of the model rather than with the difficulty of the solve, which is exactly why it is invisible for **sslp** and dominant for **farmer**.

The per-cylinder table shows the PH hub and the lagrangian spoke agreeing closely within every case, as they should, since they solve the same subproblems with different objectives. The **xhatshuffle** spoke is consistently the most Python-heavy of the three, and the gap widens as the hub’s own Python

Case	Reps	Wall (s)	Python (%)	Native (%)	System (%)	Overhead
farmer3	3	5.3	16.2 (14.4–17.6)	70.8	13.0	2.36×
farmer60	3	43.4	71.3 (70.3–72.4)	16.2	12.4	1.55×
farmer240	3	82.9	76.1 (75.2–77.0)	11.8	12.2	1.42×
farmer240_bun10	3	20.9	62.6 (61.9–63.2)	28.6	8.8	1.48×
sslp_15_45_10	3	74.0	8.7 (8.5–9.0)	89.3	2.0	1.09×
sslp_15_45_15	3	104.8	9.6 (9.4–9.9)	88.5	1.9	1.11×
sslp_15_45_15_bun3	3	73.7	12.5 (12.3–12.7)	86.1	1.4	0.99×
sslp_5_25_50	3	74.0	58.1 (58.0–58.3)	37.1	4.7	1.22×

Table 1: Time split by case, averaged over repetitions, with the min–max range over repetitions shown for the Python percentage. Percentages are of the time Scalene attributed to a source line (91.1–99.2% of wall time here); wall time is the maximum over ranks. The overhead column is profiled wall time divided by unprofiled wall time for the same case; because scalene’s instrumentation cost lands mostly on Python, it inflates the Python column.

Case	PH hub	lagrangian	xhatshuffle
farmer3	14.1 (12.7–15.5)	18.6 (17.5–20.5)	15.9 (12.6–18.4)
farmer60	72.0 (71.3–72.9)	72.6 (70.4–73.9)	69.3 (68.6–70.4)
farmer240	75.9 (75.5–76.4)	77.7 (76.6–78.6)	74.6 (73.6–76.1)
farmer240_bun10	62.3 (61.1–63.4)	64.8 (64.1–65.7)	60.8 (60.2–61.5)
sslp_15_45_10	6.8 (6.5–7.3)	6.3 (6.1–6.5)	13.0 (12.8–13.2)
sslp_15_45_15	7.2 (6.7–7.5)	6.7 (6.3–7.0)	14.9 (14.6–15.3)
sslp_15_45_15_bun3	5.1 (5.0–5.3)	7.1 (6.9–7.2)	25.3 (25.0–25.6)
sslp_5_25_50	48.6 (48.4–48.8)	48.8 (48.1–49.8)	78.1 (77.2–79.2)

Table 2: Python percentage by cylinder: mean over repetitions with the min–max range in parentheses.

share falls: 6.8% against 13.0% for `sslp_15_45_10`, and 5.1% against 25.3% for `sslp_15_45_15_bun3`. This is expected rather than anomalous. The `xhatshuffle` spoke is looking for a feasible incumbent rather than solving the subproblem to optimality, so much of its work is Python bookkeeping — shuffling the scenario order, fixing candidate nonanticipative values, and checking the result — with comparatively little time left inside the solver. It is a small fraction of total work, since it is one rank of three, but it is worth knowing that the cylinders are not interchangeable for this measurement.

2.1 Bundles

Bundling gives the solver more work per call, so it is the natural response to a high Python fraction. It helps, but not reliably, and it is worth being clear about why.

For `farmer`, bundling ten scenarios per subproblem takes the Python share from 76.1% down to 62.6% and cuts wall time by a factor of four, from 82.9 to 20.9 seconds. Fewer, larger solves means fewer round trips through Pyomo, and the wall-clock gain is large. The Python share nonetheless stays high, because a bundle of ten `farmer` scenarios is still a trivial LP.

For `sslp_15_45_15`, bundling three scenarios per subproblem moves the Python share the other way, from 9.6% up to 12.5%, while also cutting wall time from 104.8 to 73.7 seconds. Both numbers moved in the direction that makes sense once the effect is separated into its two parts: bundling reduced the total solver work more than it reduced the total Python work, so Python’s *share* rose even though the run got faster.

So bundling should be judged on wall time, where it won in both cases, and not on the Python fraction, which it can move either way. A high Python fraction is a symptom worth investigating, but driving it down is not itself the objective.

2.2 Caveats

Scalene’s instrumentation is not free, and its cost falls mostly on Python, so these Python percentages are upper bounds. The overhead column in Table 1 is the ratio of profiled to unprofiled wall time for the same case, and it lines up with the Python column: the solver-bound `sslp` cases run at 0.99–1.11×

essentially unaffected, while the Python-heavy farmer cases run at $1.42\text{--}1.55\times$. So the true Python fractions for the farmer cases are somewhat lower than the table reports. The exception is **farmer3** at $2.36\times$, which is not a Python-share effect at all: the profiler's fixed startup cost is simply large next to a two-second run, which is another reason not to trust that row for anything but its variability.

The gap between the two regimes is far too large to be an artifact of this overhead — 9% against 76% will not be closed by a factor of 1.5 — but the individual percentages should be read as approximate.

Two further limits are worth stating. These are Pyomo models and the Pyomo time is included in the Python total, so a good deal of what is labelled Python here is Pyomo rather than mpi-sppy. And all of this is one machine, one solver, and three repetitions per case.

2.3 A note on non-persistent solvers

All of the above uses `gurobi_persistent`. If you use a non-persistent interface instead — `--solver-name gurobi` is Pyomo's file-based interface, which writes an LP file and parses a solution file on every solve — then Pyomo does a great deal of extra Python work per solve, which matters most when the sub-problems are small: `sslp_5_25_50` goes from 58.1% to 75.4% Python and from 74 to 124 seconds, and even the solver-bound `sslp_15_45_10`, whose wall time barely moves, goes from 8.7% to 31.8% Python. Use a persistent interface if you have one.